

COS214 Practical 2

Courtesy of

- Patrick Simuyemba: u25632354
- Azolile Mbanga: u24820360

Task 1: Diagnosis of the broken design—

1.1

- Movement: MoveStrategy is trying to implement the Strategy design pattern, and it should rather be implemented as the State design pattern.
- Route: RouteState is trying to implement the State design pattern, and it should rather be implemented as the Strategy design pattern.
- Map: Map is trying to implement the Composite design pattern.
- Place features: The place features are trying to implement the Decorator design pattern.
- Biomes: WorldBuilder is trying to implement the Abstract Factory design pattern.
- The two labels that have been swapped is the Strategy and the State. Movement should be done with the State design pattern because an object should be able to change to a different move at runtime. The behaviour changes because the Traveller changes its state/mode. Route should be done with the Strategy design pattern because different route strategies use different algorithms to calculate the route. Choosing a route is interchangeable and changing the strategy changes how a route is calculated not the State of a trip.

1.2

- Movement: Under the State pattern, the participants are the Client, Context, State and ConcreteStateN
- Route: Under the Strategy design pattern, the participants are the Client, Context, Strategy, ConcreteStrategyN
- Map: Under the Composite design pattern, the participants are the Client, Leaf, Composite, Component
- Place features: Under the Decorator design pattern, the participants are the Client, Component, ConcreteComponent, Decorator, ConcreteDecoratorA, ConcreteDecoratorB
- Biomes: Under the Abstract Factory design pattern, the participants are the Client, AbstractFactory, AbstractProduct, ConcreteFactory, ConcreteProduct

1.3

- Wrong Pattern Choice: MoveStrategy is implementing the Strategy pattern when it should be implementing the State pattern. RouteState is implementing the State pattern when it should be implementing the Strategy pattern. The consequence in implementing this is the maintenance overhead that will come with adding new subclasses to the Strategy or State interfaces. The misuse creates unnecessary changes to existing classes when extending the system, increasing maintenance.
- Wrong UML Notation: The solid arrow used for the aggregation ‘uses’ relationship in the MoveStrategy relationship is incorrect because it is meant to be a solid line without an arrow. The <<Decorator>> label is not in a class diagram box, so it is not labelling anything. The maintenance consequence is that the intended pattern/class relationship is unclear.
- Broken ownership or missing virtual destructors: Location has no virtual destructor, so deleting a derived object through a Location pointer can result in undefined behaviour and resources owned by the derived class may not be correctly destroyed. The inheritance relationships with Location are flattened. That creates a maintenance overhead to manage each subclass with each added tag they have in their class name.
- Excessive Coupling: GameManager is dependent on all 5 subsystems. Changes to any subsystem would require modifications to GameManager, increasing maintenance and reducing extensibility. The separation of responsibilities has been violated. WorldBuilder is tightly coupled with the concrete biome products because its make function contains a giant switch that knows which concrete object to create.

Adding a new biome product to the subsystem would require modifications to the WorldBuilder class, increasing maintenance and violating the goal of separating creation from the client.

1.4

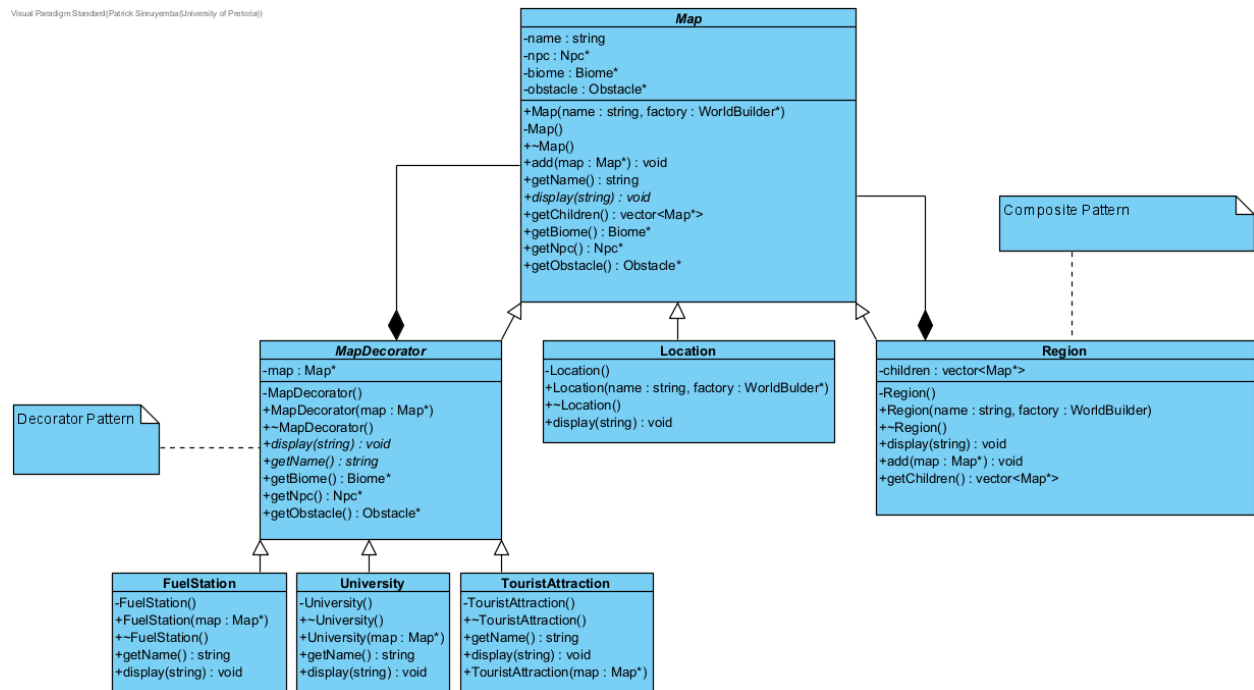
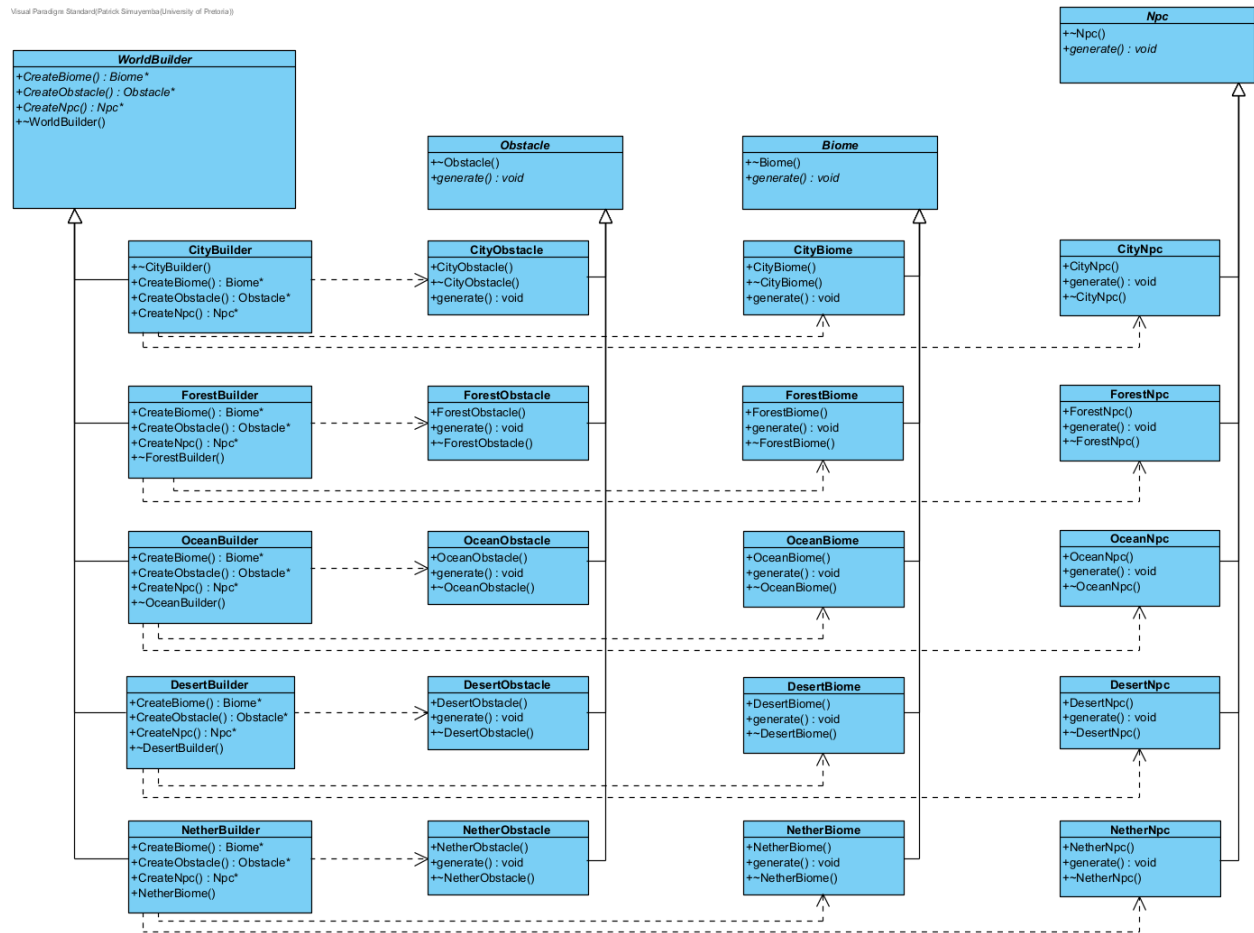
- The GameManager class is the class that has been made to know everything. Coupling is how much classes depend on each other, and design patterns help to reduce high coupling and unnecessary dependencies by separating responsibilities and allowing components to interact through abstractions. GameManager should not centralise all the behaviour of the system. Each subsystem should manage what belongs to its responsibility. The Region class should own both other regions and the location objects the as Composite participant in that subsystem. The WorldBuilder class should use the AbstractFactory structure, with a ConcreteFactory for each biome responsible for creating that biome's related products. Movement and Route-finding should have their own Context classes with Traveller and Trip acting as those Context classes.

1.5

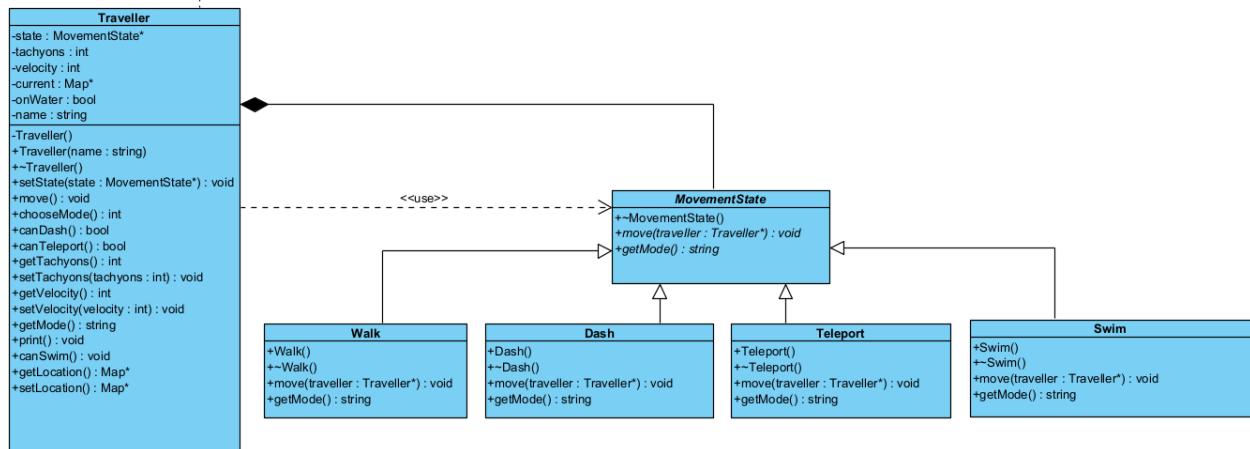
- Arrows are overlapping the class diagrams and that interference makes it hard to follow where the relationships are and it is confusing to read the names of certain classes. The classes are arranged in a tightly packed manner on one side and then there is lots of white space and another section of the system. It is difficult to identify each subsystem because of how closely packed certain sections are. To improve the diagram, one could separate each subsystem to be evenly spaced around the central GameManager class. I would route the arrows around classes/components and not allow them to overlap with any other component.

Task 2 : Redesign

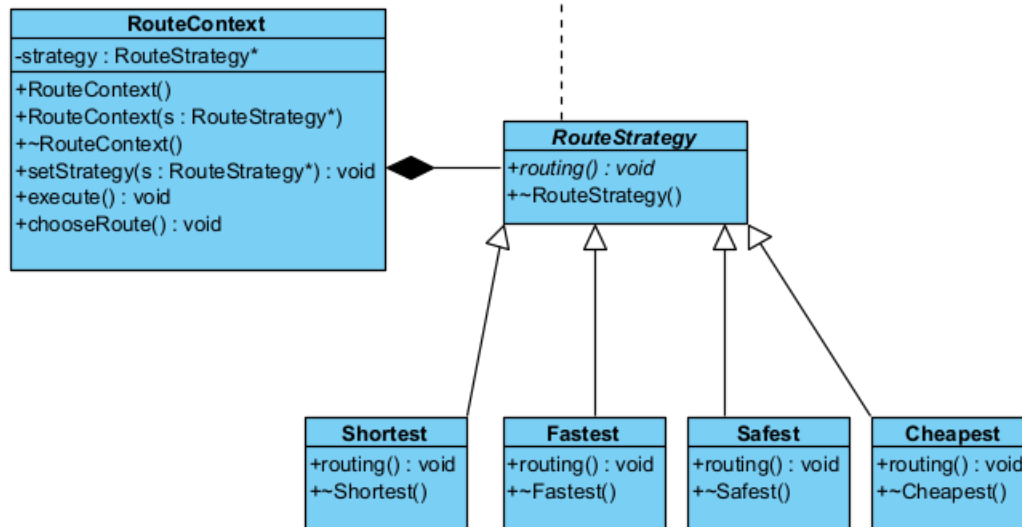
2.2 Diagram

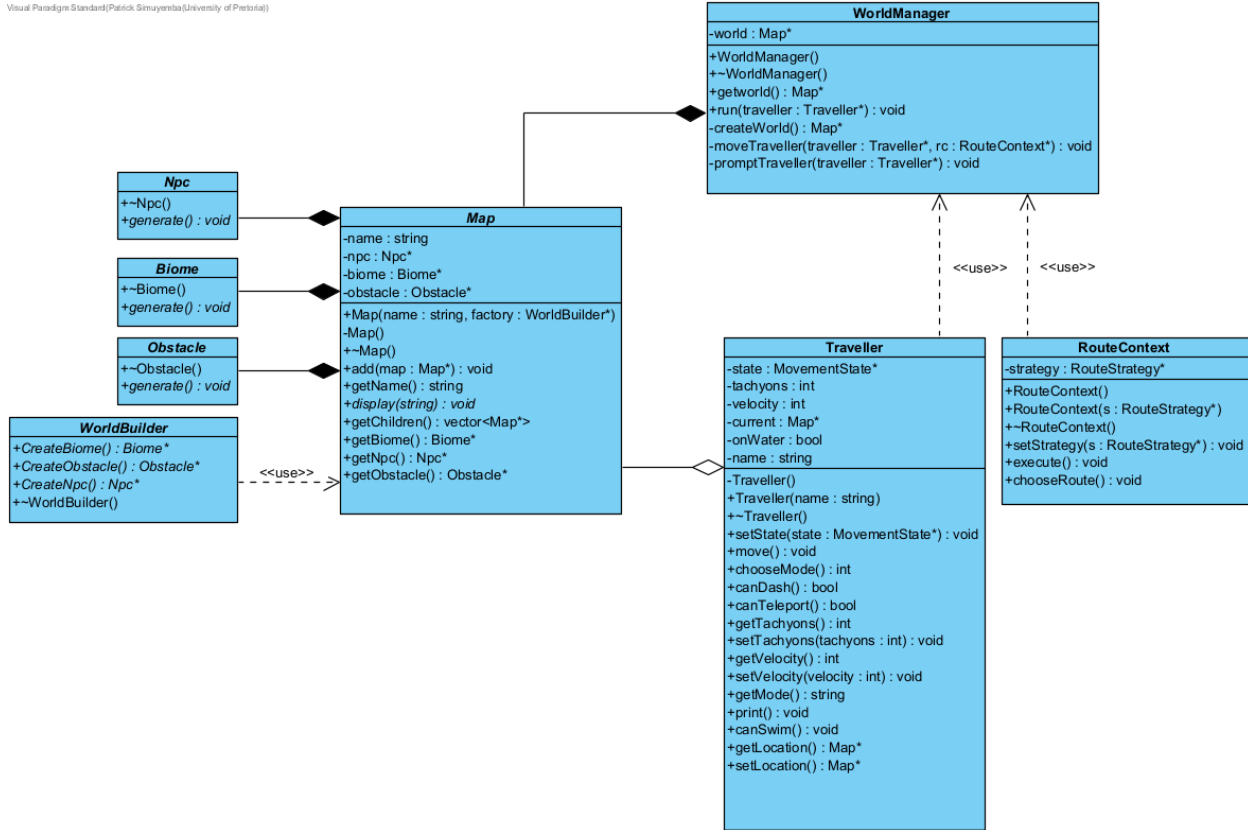


State Pattern



Strategy Pattern





2.2 Design rationale

Pattern: Decorator Pattern

Participants:

- Component: **Map**
- ConcreteComponent: **Location**
- Decorator: **MapDecorator**
- ConcreteDecorator: **University**, **FuelStation**, **TouristAttraction**

Decision

- **MapDecorator** owns and deletes the wrapped **Map***. Features can stack in any order without making a new class for every combination.

Pattern: Composite Pattern

Participants

- Component: **Map**
- Composite: **Region**
- Leaf: **Location**

Decision

- **Region** owns and deletes its collection of **Map*** pointers. Deleting the root frees the whole map. Clients use one **Map** interface for both locations and regions.

Pattern: State Pattern Participants

- State: **MovementState**
- Context: **Traveller**

- ConcreteState: Walk, Dash, Teleport, Swim

Decision

- Traveller only talks to abstract MovementState*. Each mode is its own class, so movement can change at runtime with no big switch in the context.

Pattern: Strategy Pattern Participants

- Strategy: RouteStrategy
- Context: RouteContext
- ConcreteStrategy: Fastest, Safest, Shortest

Decision

- RouteContext owns a pointer to RouteStrategy*.
- Route algorithms are swappable and we can add more algorithms without changing the RouteContext class because we don't have a switch or if statement for deciding the route in the context, the context just needs to execute strategy->routing() which will run the appropriate concrete algorithm.

Pattern: AbstractFactory Method Participants

- AbstractFactory: WorldBuilder
- ConcreteFactory: OceanBuilder, DesertBuilder, ForestBuilder, CityBuilder, NetherBuilder
- AbstractProduct: Npc, Biome, Obstacle
- ConcreteProduct:
 - DesertBiome, DesertObstacle, DesertNpc
 - OceanBiome, OceanObstacle, OceanNpc
 - ForestBiome, ForestObstacle, ForestNpc
 - CityBiome, CityObstacle, CityNpc
 - NetherBiome, NetherObstacle, NetherNpc

Decision

- Each biome factory only creates its own matching products (e.g. desertNPC + desertBiome + desertObstacle together). Clients use WorldBuilder and abstract products, so they don't invent mismatched combos like a desert NPC with an ocean obstacle. Adding a biome means one new factory and its products only.

2.3 Additions

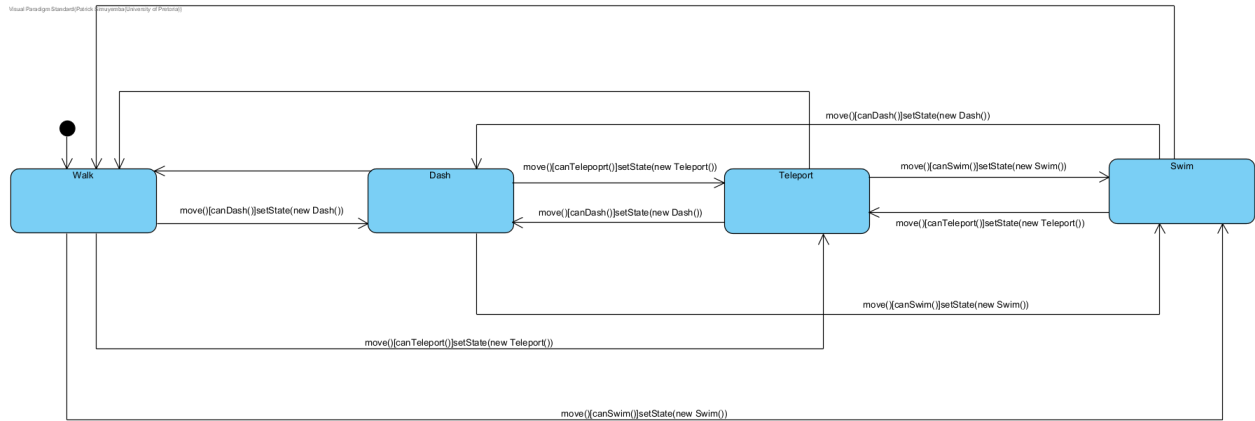
- Teleport (State)
 - New concrete MovementState for travelling by disappearing and reappearing in another location.
 - Only a new state class (and its transition rules) is added. Traveller and the other modes stay unchanged.
- safest (Strategy)
 - New concrete RouteStrategy that prefers safer routes.
 - Only a new strategy class is added; RouteContext still calls the same abstract interface.
- NetherBiome (AbstractFactory)
 - New NetherBuilder that creates NetherBiome, NetherNpc, and NetherObstacle.
 - Only that factory and its products are added; existing biome builders stay unchanged.

Task 3 : Movement with State

3.1 Completed

3.2 Completed

3.3 State Diagram



Task 4 : Route finding with Strategy

4.1 Completed

4.2 Completed

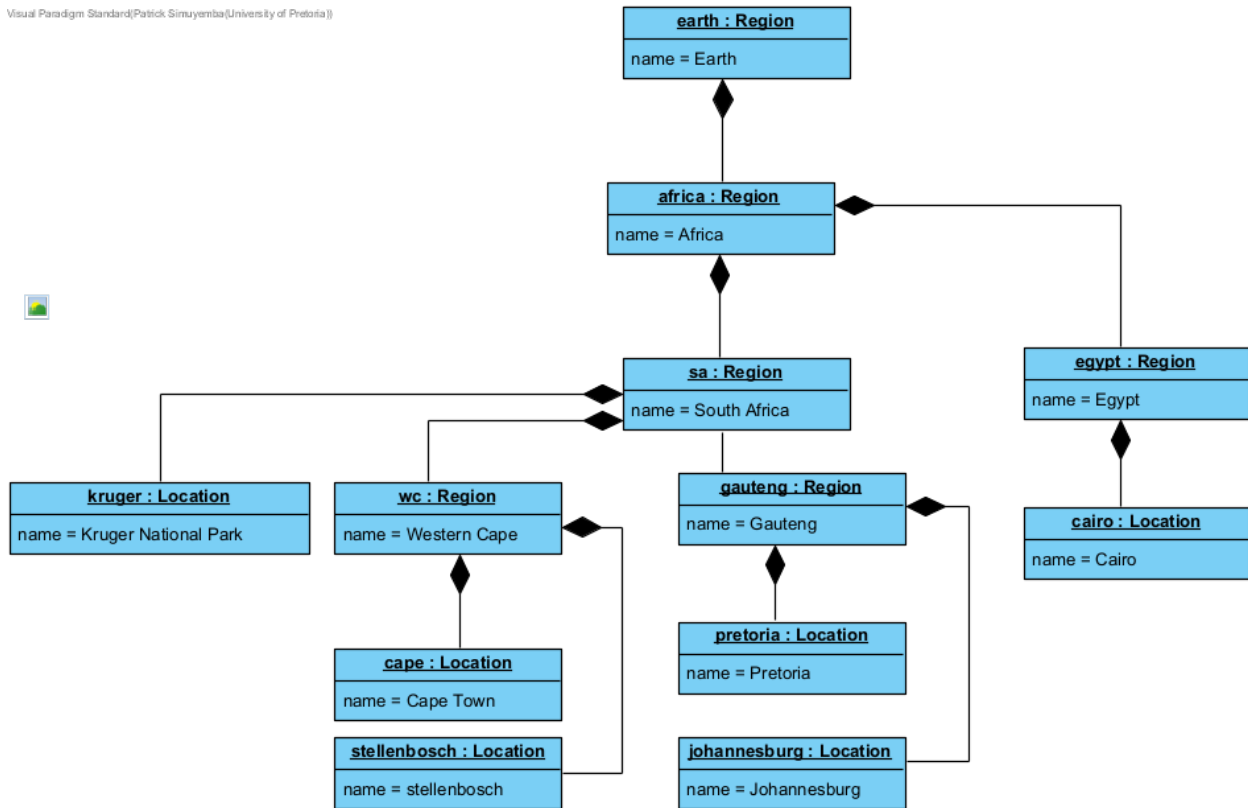
4.3

- The Strategy's intent is to have a family of algorithms and encapsulate each one, and make them interchangeable. It lets the algorithm vary independently from clients that use it. The State's intent is to allow an object to alter its behaviour when its internal state changes. The object will appear to change its class. Routes are Strategy because different route strategies use different algorithms to calculate the route and choosing a route is interchangeable. Movement is State because an object should be able to change to a different move at runtime. The behaviour changes because the Traveller changes its state/mode. The class diagrams are not interchangeable even though they look alike. They differ in their intent and usage. For the State, the ConcreteState controls the changes where an object will appear to change its class while for the Strategy, the Client will control the change in algorithm used depending on the Context.

Task 5 : The world map with Composite

5.1 Completed

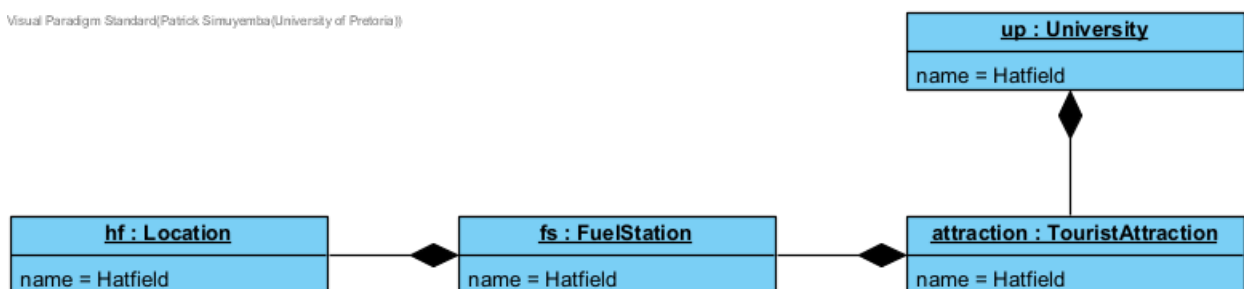
5.2 Object Diagram



Task 6 : Place features with Decorator

6.1 Completed

6.2 Object Diagram



Task 7 : Biomes with Abstract Factory, and intergration

7.1 Completed

7.2 Completed

7.3 Completed